HeyMax Analytics Project Documentation

Table of Contents

Project Overview

This project sets up an end-to-end open-source data and analytics pipeline for HeyMax with an interactive analytics dashboard built using Streamlit, DuckDB, and dbt to analyze user lifecycle metrics over time. It supports monthly, weekly, and daily views with charts, retention triangle tables, KPIs, filters, and LLM-based insights.

Access the live dashboard here: https://heymax-analytics-dev.streamlit.app/

Features

- 📅 Toggle between Monthly, Weekly, and Daily metrics
- 📈 Charts for:
  - New, Resurrected, Retained, Churned, and Active Users
  - Stacked bar + line charts (Quick Ratio, Retention Rate)
- 📐 Retention Triangle (monthly, weekly, and daily cohorts)
- 🎯 KPI cards for the latest period
- 🎛️ Filters: date range
- 💬 Ask your data (powered by OpenAI's GPT-4)
- ⬇️ Download filtered report as CSV

Setup

To setup the project, please refer to the README.md file:
https://github.com/SANCHIT-GARG/heymax-analytics/blob/main/README.md

About the Data

- event_time: timestamp, needs to be truncated to month
- user_id: string, uniquely identifies user
- event_type: e.g., miles_earned, miles_redeemed, share
- transaction_category: dining, ecommerce, flight, etc.
- miles_amount: some events may not have this (e.g. share)
- platform, utm_source, country: standard metadata fields

Technology Stack

DBT

DBT enables SQL-based transformation with support for testing, documentation, and scalable DAGs. Use '**dbt docs generate**' to view model docs. Parquet storage, partitioning, and sort keys improve performance in DuckDB.

DuckDB
DuckDB is an OLAP SQL engine ideal for analytical queries. It supports in-memory operations, Parquet IO, and integrates with cloud environments. Query via CLI (**'duckdb -ui'**) or embed in Python.

Streamlit
Streamlit builds interactive Python dashboards rapidly. It supports Python-native components, real-time reactivity, and deploys easily on Streamlit Cloud.

GitHub Deployment
- CI: Executes dbt runs/tests on every commit, emails logs, updates DuckDB, and runs daily at 8am. (the runs can be configured based on the requirements)
- CD: Uses Streamlit Cloud linked to the repo for zero-touch deployments.

Design Considerations

1. Data Modeling and Transformation Logic
Raw events are loaded from CSV and staged via `stg_raw_events`. Business logic lives in intermediate models like `dim_users`, `fct_events`, and `user_lifecycle_*`, leveraging incremental materialization and delete+insert logic. Partitioning and sort keys enhance query speed.

2. Dashboard Design and Metric Clarity
The dashboard offers toggles for daily/weekly/monthly views, chart summaries, cohort tables, and KPIs. Users can filter by date and export CSVs. LLM integration allows natural language queries.

Example questions for LLM:
- What month had the highest churn rate?
- Compare new users vs resurrected users in April.
- Why did quick ratio drop in May?
- How many users were retained after 3 months from the March cohort?

3. Code Readability, Structure, and Documentation
The project uses a clean structure with layered SQL folders (staging, intermediate, growth) and modular Python code organized into Streamlit tabs. Metadata and tests are defined via schema.yml.
CI is set up using GitHub Actions (.github/workflows/dbt_run.yml) to run dbt, update heymax.duckdb, and send email notifications on success/failure using GitHub secrets (EMAIL_USER, EMAIL_PASSWORD). It runs on every commit and daily at 8 AM.
CD is handled via Streamlit Cloud, auto-deploying the app (dashboard.py) on every push to main.

4. Design Tradeoffs, Testing, and Scaling Strategy

DuckDB offers blazing speed for local analytics but isn't built for high-concurrency and access based control. Dbt tests ensure data quality. The modular structure supports easy migration to BigQuery or Snowflake.

DBT Modeling Strategy

This project uses a layered dbt modeling approach based on the Medallion Architecture. It transforms raw event data into actionable insights across daily, weekly, and monthly levels.

Model Summary

Model Name	Layer	Materialization	Strategy	Storage Format
stg_raw_events	Staging	Table	Full refresh	Parquet
stg_events	Staging	Table	Full refresh	Parquet
dim_users	Intermediate	Incremental	delete+insert	Parquet
fct_events	Intermediate	Incremental	delete+insert	Parquet
user_lifecycle_*	Intermediate	Incremental	delete+insert	Parquet
growth_metrics_*	Growth	View	-	-
retention_triangle_*	Growth	View	-	-

Layering and Its Benefits

This dbt project follows the Medallion Architecture:

1. Staging Layer
    - Purpose: Clean raw source data
    - Tables: stg_raw_events, stg_events
    - Characteristics: Simple SELECT + CAST, no business logic
    - Why Tables? DuckDB benefits from persisted Parquet tables for fast reads.
2. Intermediate (Core) Layer
    - Purpose: Central logic and transformations (fact/dim)
    - Tables: dim_users, fct_events, user_lifecycle_*
    - Why Incremental? For scalability and pipeline efficiency
    - Why delete+insert? Ensures upserts during recomputation
3. Growth Layer
    - Purpose: Analytical outputs and aggregations
    - Views: growth_metrics_*, retention_triangle_*
    - Why Views? These depend on constantly changing input and should reflect live business metrics

Analysis and Insights

Assumptions

- Users are uniquely identified by `user_id`.
- The `event_time` field is the basis for all time-based aggregations (daily/weekly/monthly).
- Metrics are aggregated at the user level, not event count level.

Metric Definitions

- New Users: Users active for the first time in the selected period.
- Resurrected Users: Users who were inactive in the previous period but became active again.
- Retained Users: Users active in both the current and previous period.
- Churned Users: Users who were active previously but inactive in the current period.
- Active Users: Total users who performed at least one action in the period.
- Retention Rate: Percentage of retained users from the previous period.
- Quick Ratio: (New + Resurrected) / Churned — indicates growth vs loss balance.

Insights

- **Limitations in New User Insights Due to Data Framing**
  The observed decline in new user acquisition starting March 2025—visible in both weekly and monthly views—should be interpreted with caution. The dataset spans only three months, which is a relatively short timeframe to draw strong conclusions. Additionally, the logic currently used to define "new users" considers any user seen for the first time in this dataset as new. This can lead to misleading inferences once the entire known user pool has been seen at least once. The lack of new users beyond a certain point is not necessarily a reflection of acquisition failure, but rather of dataset boundaries and how identity is framed. To gain stronger confidence, we would benefit from either historical context or continued data collection over the coming months.
- **Unusually Perfect Retention Suggests Potential Simulation or Definition Gap**
  Weekly cohorts from March 9 and March 16 show a 100% retention rate, and daily cohorts consistently report full day-0 retention. While promising on the surface, such flawless retention is rare in real-world scenarios and suggests either simulation or a potential misalignment in how user activity/inactivity is defined. In most practical cases, some degree of churn is expected—even among early adopters. This highlights the need to revisit how activity is being measured and validate if this aligns with the business's operational definition of an "active" user.

- **Cohort Retention Drop Is Artificial—A Byproduct of Augmented Analysis**
  The observed retention drop for the March cohort by month four is not from the actual user data but stems from artificially extending the timeline to construct a cohort matrix. This augmentation was necessary to build a complete cohort view, but the resulting retention decline is not organically present in the data. In fact, across all original data points, retention appears unusually consistent, which once again suggests either simulated data or the need to review how we're calculating lifecycle metrics in context of the business use case.
- **Quick Ratio Trends Suggest Healthy Growth—With the Caveat of Incomplete Inputs**
  Across the daily granularity, the quick ratio remains consistently above 1—indicating that user growth (new + resurrected) exceeds churn. While this is typically a positive signal, it should be interpreted in the context of the dataset's constraints. The definitions of "new" and "resurrected" users need scrutiny to ensure they reflect meaningful user behavior, especially when data completeness is in question. Without this validation, quick ratio patterns may present a skewed picture of growth health.
- **Acquisition Spikes Point to Irregular Patterns—But Must Be Verified for Authenticity**
  Daily user trends reveal sharp acquisition spikes—such as 52 users on March 8—followed by long periods of inactivity, including no new users from March 14 onward. While this hints at unreliable or burst-driven growth mechanisms, it again raises the question of whether this pattern is real or shaped by dataset simulation or framing artifacts. For any sustainable growth analysis, we must establish that acquisition signals are grounded in accurate and complete tracking logic.

Follow-up Questions

- What specific changes in marketing strategy, product rollout, or external conditions occurred around March 2025 that could explain the observed decline in new user acquisition?
- Are the high-retention cohorts from early March linked to specific acquisition sources, campaigns, or onboarding flows that could be analyzed and replicated?
- Should the definitions of "new," "active," or "churned" users be revised to align more closely with business logic and better reflect true customer behavior?
- Given the limited timeframe and unusually high retention patterns, is the dataset complete and representative of real user behavior, or should its reliability be validated before basing decisions on these trends?

Future Enhancements and Scaling Roadmap

Upgrade Area	Suggested Tools	Benefit
Data Warehouse / Lakehouse	Redshift , BigQuery, Snowflake, Databricks	Scales to handle large datasets, high concurrency, and complex analytics
Data Orchestration	Airflow, dbt Cloud, Dagster	Automates dbt runs, testing, dependency management, and alerting
Dashboarding	Looker, Tableau, Metabase	Enables production-ready dashboards with fine-grained role-based access control (RBAC)
Streaming Ingestion	Kafka, Pub/Sub	Supports near real-time data processing and dashboard updates

Planned UI enhancements

- Add **granular UTM tracking, filters for country** for better user acquisition insights
- Have a **custom period** for churn, retention, resurrected and retained users based on the requirements of business.
- Implement **role-based access and authentication** in the Streamlit app using streamlit-authenticator
- Enhance the LLM interface with **chat history and intelligent prompt suggestions**